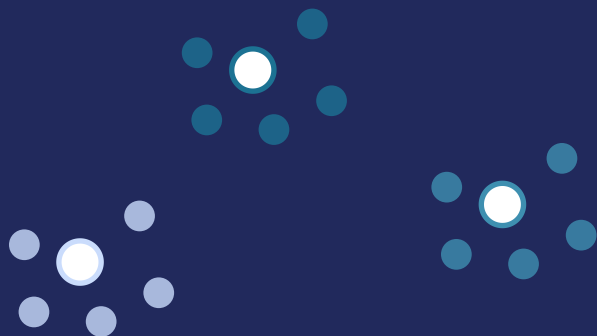


UNSUPERVISED ML · DAY 1

K-Means Clustering

From random centroids to customer personas — a first-principles walkthrough

Algorithm mechanics · Initialization & convergence · Limitations · Choosing K · Cluster profiling



What we'll walk through

1

How K-Means actually works

Initialization, assignment, and the mean-update loop

2

Convergence & bad starts

Stopping rules, local minima, and K-Means++

3

Scaling to big data

Mini-batch K-Means and the `n_init` / inertia trick

4

Where it breaks

Data geometry, sparsity, multicollinearity, dimensionality

5

Choosing K

Elbow method and Silhouette score, side by side

6

Making it useful

Turning cluster numbers into business personas

Why this feels different from Day 1 of Supervised

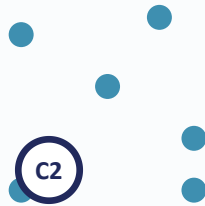
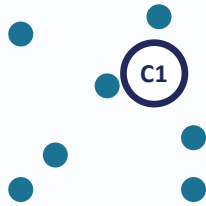
Supervised (what you know)

- Has labels: X_{train} , y_{train}
- Metric: accuracy / RMSE against ground truth
- Goal: predict a known target
- You already mastered: EDA, feature engineering, preprocessing, hyperparameter tuning

Unsupervised (starting today)

- No labels: only X , no y at all
- Metric: inertia / silhouette — no ground truth exists
- Goal: discover hidden structure
- New muscle to build: judging "good clusters" without anyone telling you the right answer

Initialize centroids, then assign nearest points



Random initial centroids (not real data points) + nearby points assigned by distance

Step 1 — Initialize

Drop K centroids at random coordinates. They are guesses, not real data points.

Step 2 — Assign

For every data point, measure distance to every centroid (Euclidean). Assign the point to its nearest centroid.

Move centroid to the mean, then repeat



Centroid shifts to the true center of its assigned points

Step 3 — Update

New centroid = mean of x , mean of y of its members. The mean is the mathematically optimal point — it minimizes total squared distance to the group.

Step 4 — Repeat

Moved centroids change who's nearest — some points switch clusters. Reassign, then re-average. This loop continues until nothing changes.

Assign → Update → Assign → Update → ...

How does the algorithm know it's done?

1

No reassignment

On a full pass, zero data points switch clusters.

Cluster membership is fully stable.

2

Centroid shift < tol

Distance each centroid moved is smaller than a threshold (e.g. 0.0001).

"Effectively stable" — an engineering choice (sklearn's tol), not a derived constant.

3

Max iterations hit

A safety cap (e.g. 300) stops the loop even without full convergence.

Prevents runaway loops on stubborn or noisy data.

Bad random starts → bad local minima

Plain random init



Two centroids can land close together — one region gets over-served, another under-served

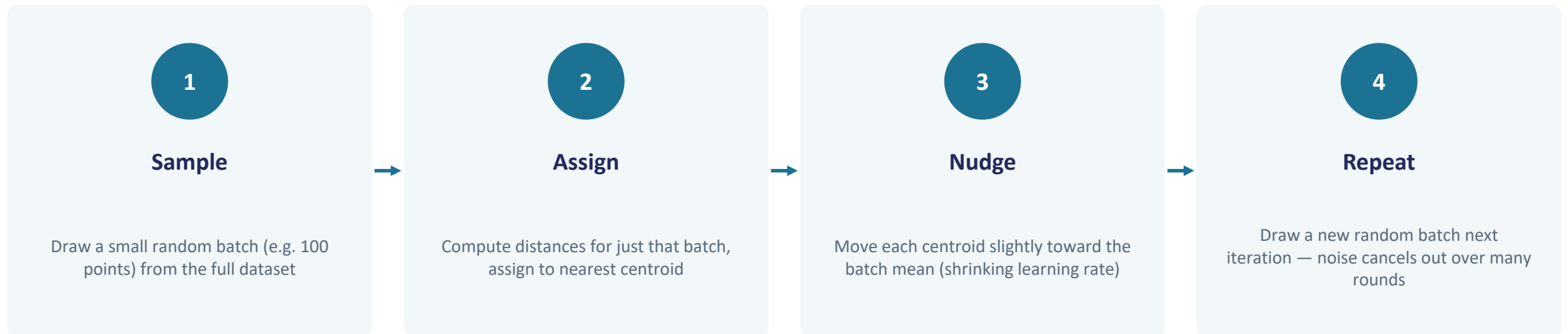
K-Means++ init (sklearn default)



First centroid random, next ones chosen with probability proportional to distance from existing centroids — spreads them out deliberately

Mini-Batch K-Means: same idea, sampled data

Full K-Means recomputes the mean using the entire dataset every iteration — expensive at scale. Mini-batch trades a little precision for a lot of speed.



Why it still converges: each batch is noisy, but the average effect across many batches pulls every centroid toward the true center of its region — like a coin settling near 50% over many flips, not few.

n_init: run it 10 times, keep the tightest fit

Inertia (WCSS)

$$\Sigma (\text{distance from point to its centroid})^2$$

Lower = tighter, more compact clusters. This is exactly what K-Means minimizes.

What n_init=10 actually does:

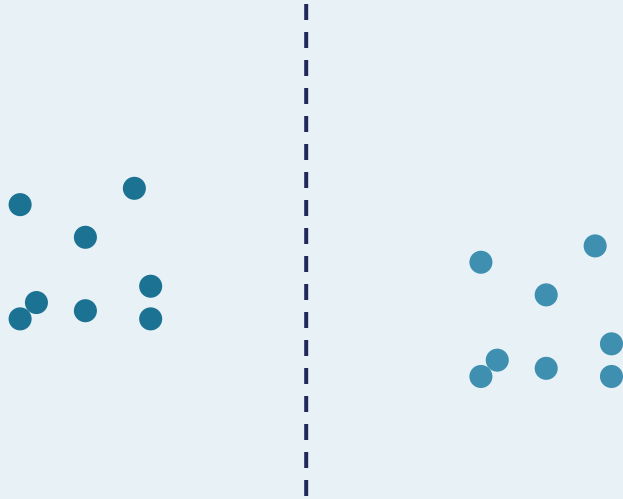
- Run 1: random start A → loop to convergence → inertia A
- Run 2: random start B (independent) → loop to convergence → inertia B
- ... 10 completely separate full runs in total

Why it's necessary: bad random starts can lock K-Means into a suboptimal arrangement — it can only ever nudge centroids to reduce local distance, never "jump" to a totally different layout mid-run.

Key clarification: each of the 10 runs is a complete, independent run of assign → update → ... → convergence. Not 10 iterations — 10 full executions, each with its own many internal iterations.

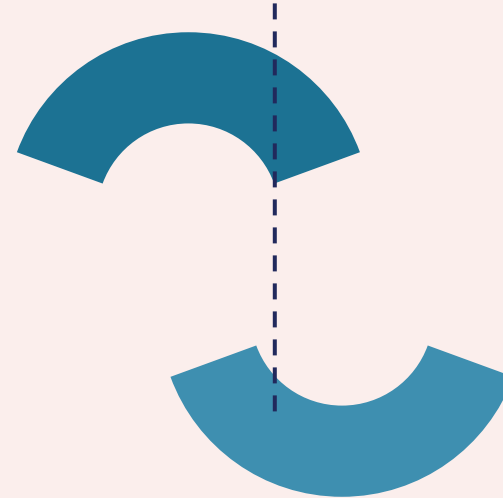
K-Means only draws straight-line boundaries

Spherical data — K-Means wins



Clean convex "blob" clusters → straight Voronoi boundary matches reality

Crescent moons — K-Means fails



A straight cut slices through both moons — mixes points from each into the wrong group

What kind of data does K-Means actually like?

OK

Linear / convex-separable structure

Only draws straight-line boundaries between centroids

X

Non-linear relationships (moons, rings, spirals)

Cannot bend a boundary — slices curved groups apart

X

Sparse data (e.g. TF-IDF text vectors)

Distances converge in high-dim sparse space; mean of a sparse vector isn't sparse

X

Multicollinearity (correlated features)

Correlated features get implicitly double-counted in distance

X

High dimensionality / many features

Curse of dimensionality — all pairwise distances start looking equal

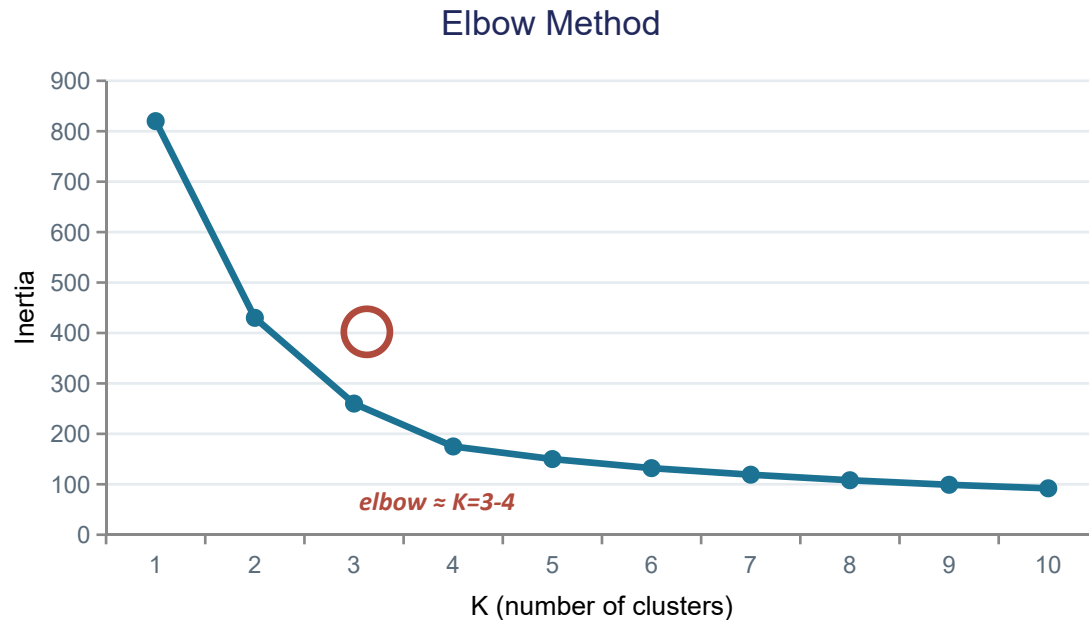
Fix: Scale → PCA → K-Means. PCA reduces dimensions, decorrelates features, and restores meaningful distances.

Six limitations, and who fixes each one

Limitation	The issue	Fixed by
Manual K	Must choose K upfront	DBSCAN / HDBSCAN
Outlier sensitivity	Mean is easily skewed by extreme points	DBSCAN (flags noise explicitly)
Spherical assumption	Fails on crescents, rings, elongated shapes	DBSCAN (arbitrary shapes)
Feature scale	Large-range features dominate distance	StandardScaler (non-negotiable)
Init sensitivity	Random start → local minima	K-Means++ (sklearn default)
Differing density/size	Splits big clusters, merges sparse ones	GMM (soft, probabilistic)

Every "fix" trades in a new weakness — K-Means is fast (~linear time); DBSCAN and HDBSCAN cost more compute as data grows.

Elbow Method + Silhouette Score, together



Silhouette Score

$$\text{Score} = (b - a) / \max(a, b)$$

a = avg. distance to points in the SAME cluster

b = avg. distance to points in the NEAREST other cluster

+1

Excellent — close to own group, far from others

0

On the boundary between overlapping clusters

-1

Likely assigned to the wrong cluster

Use both: pick the K where Silhouette peaks near the elbow point.

Cluster labels → business personas

groupby('cluster_label').mean() turns raw numbers into a profile per group:

Cluster	Avg Age	Avg Income	Avg Spend	Persona	Action
0	45	\$88,000	18	Frugal High-Earners	Premium investment / savings ads
1	25	\$30,000	79	Trendseekers	Budget fashion & fast discounts
2	32	\$85,000	88	VIP Big Spenders	Loyalty rewards & exclusive perks

Same three steps every time: attach the cluster label to each row → aggregate feature means per cluster → name the pattern in human terms.

NEXT UP

From centroids to density



DBSCAN

Finds K automatically, traces curved shapes, flags outliers as noise instead of forcing them into a group



GMM

Soft, probabilistic assignment — handles overlapping clusters and differing densities better than hard means



HDBSCAN

DBSCAN's density-threshold tuning, automated — the practical upgrade for messy real-world data

Day 1 complete: algorithm mechanics, convergence, initialization, scale-up, data-fit, K-selection, and profiling.